

SKILL WEEK-1

Session 1

Task1: To Install Linux VM, Configure GCC, Setup Git Repository, Create Project Structure, Understand Shell Architecture, Build Initial Make file.

Code:

```
GNU nano 7.2 Makefile *
CC = gcc
CFLAGS = -Wall -Wextra

all: main

main: main.c
    $(CC) $(CFLAGS) main.c -o main

clean:
    rm -f main
```

```
#include <stdio.h>

int main() {
    printf("Hello, OSSP Skill Week 1!\n");
    printf("GCC compilation is working successfully.\n");

    return 0;
}
```

Outputs:

```
make: Nothing to be done for 'all'.
avulasanjana@SanjanaReddy:~/ossp_2520090159/skill/Week1/Task1$ ./main
Hello, OSSP Skill Week 1!
GCC compilation is working successfully.
```

```
avulasanjana@SanjanaReddy:~/ossp_2520090159/skill/Week1/Task1$ nano main.c
avulasanjana@SanjanaReddy:~/ossp_2520090159/skill/Week1/Task1$ gcc main.c -o
main
avulasanjana@SanjanaReddy:~/ossp_2520090159/skill/Week1/Task1$ ./main
Hello, OSSP Skill Week 1!
GCC compilation is working successfully.
```

Verifying linux environment by uname-a:

```
avulasanjana@SanjanaReddy:~$ uname -a
Linux SanjanaReddy 6.18.33.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC
Thu Jun 18 21:54:43 UTC 2026 x86_64 GNU/Linux
```

Checking GCC C compiler:

```
avulasanjana@SanjanaReddy:~$ gcc --version
gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

Git version:

```
avulasanjana@SanjanaReddy:~$ git --version
git version 2.53.0
```

Observation: I learned how to work with the Linux environment and verify system tools such as GCC and Git. I learned how to compile and execute a C program using GCC and understood the purpose of a Makefile for automating the compilation process. I also learned how Linux development projects can be organized and built using standard compiler and build commands.

Task2: To Analyse process abstraction, execute fork(), understand exec() family, analyse parent-child relationships, inspect process tree, practice system call tracing.

Code:

```
GNU nano 7.2 skill_week1_task2.c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t pid;

    printf("Parent Process Started\n");
    printf("Parent PID: %d\n", getpid());

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid == 0) {
        printf("\nChild Process\n");
        printf("Child PID: %d\n", getpid());
        printf("Parent PID: %d\n", getppid());

        printf("Executing ls using exec()\n");

        execlp("ls", "ls", "-l", NULL);

        perror("exec failed");
        exit(1);
    }
    else {
        printf("\nParent Process\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);
        printf("Parent waiting for child...\n");

        wait(NULL);

        printf("Child process completed.\n");
    }

    return 0;
}
```

Output:

```
avulasanjana@SanjanaReddy:~$ nano skill_week1_task2.c
avulasanjana@SanjanaReddy:~$ gcc skill_week1_task2.c -o skill_week1_task2
avulasanjana@SanjanaReddy:~$ ./skill_week1_task2
Parent Process Started
Parent PID: 507

Parent Process
Parent PID: 507
Child PID: 508
Parent waiting for child...

Child Process
Child PID: 508
Parent PID: 507
Executing ls using exec()
total 248
-rw-r--r-- 1 avulasanjana avulasanjana 109 Aug 7 05:17 File1.txt
-rw-r--r-- 1 avulasanjana avulasanjana 109 Aug 7 05:19 File2.txt
-rw-r--r-- 1 avulasanjana avulasanjana 0 Aug 5 03:41 LICENSE
-rw-r--r-- 1 avulasanjana avulasanjana 0 Aug 5 03:41 Makefile
drwxr-xr-x 4 avulasanjana avulasanjana 4096 Aug 11 06:30 OSSP
-rw-r--r-- 1 avulasanjana avulasanjana 0 Aug 5 03:41 README.md
drwxr-xr-x 2 avulasanjana avulasanjana 4096 Aug 5 03:41 assets
drwxr-xr-x 2 avulasanjana avulasanjana 4096 Aug 5 03:41 bin
-rwxr-xr-x 1 avulasanjana avulasanjana 16528 Aug 11 06:59 command_executor
-rw-r--r-- 1 avulasanjana avulasanjana 1387 Aug 11 06:59 command_executor.c
drwxr-xr-x 2 avulasanjana avulasanjana 4096 Aug 5 03:41 docs
-rwxr-xr-x 1 avulasanjana avulasanjana 16216 Aug 7 05:19 file_copy
-rw-r--r-- 1 avulasanjana avulasanjana 1113 Aug 7 05:17 file_copy.c
-rwxr-xr-x 1 avulasanjana avulasanjana 16432 Aug 13 08:28 fork_exec
-rw-r--r-- 1 avulasanjana avulasanjana 947 Aug 13 08:28 fork_exec.c
drwxr-xr-x 2 avulasanjana avulasanjana 4096 Aug 5 03:41 include
-rwxr-xr-x 1 avulasanjana avulasanjana 16192 Aug 7 05:39 keyboard_input
-rw-r--r-- 1 avulasanjana avulasanjana 1803 Aug 7 05:39 keyboard_input.c
-rwxr-xr-x 1 avulasanjana avulasanjana 16352 Aug 7 05:09 main_loop
-rw-r--r-- 1 avulasanjana avulasanjana 800 Aug 7 05:09 main_loop.c
drwxr-xr-x 2 avulasanjana avulasanjana 4096 Aug 5 03:41 obj
drwxr-xr-x 3 avulasanjana avulasanjana 4096 Aug 13 08:37 ossp_2520090159
-rwxr-xr-x 1 avulasanjana avulasanjana 16824 Aug 11 06:41 parser_double_quotes
-rw-r--r-- 1 avulasanjana avulasanjana 3934 Aug 11 06:41 parser_double_quotes.c
-rwxr-xr-x 1 avulasanjana avulasanjana 16560 Aug 11 06:34 parser_quotes
-rw-r--r-- 1 avulasanjana avulasanjana 2249 Aug 11 06:34 parser_quotes.c
-rwxr-xr-x 1 avulasanjana avulasanjana 15952 Aug 1 04:49 process
-rw-r--r-- 1 avulasanjana avulasanjana 80 Aug 1 04:48 process.c
-rw-r----- 1 avulasanjana avulasanjana 80 Aug 1 04:46 process.c.save
drwxr-xr-x 2 avulasanjana avulasanjana 4096 Aug 5 03:41 scripts
-rwxr-xr-x 1 avulasanjana avulasanjana 16304 Aug 13 10:14 skill_week1_task2
```

```
-rwxr-xr-x 1 avulasanjana avulasanjana 16304 Aug 13 10:14 skill_week1_task2
-rw-r--r-- 1 avulasanjana avulasanjana 879 Aug 13 10:13 skill_week1_task2.c
drwxr-xr-x 2 avulasanjana avulasanjana 4096 Aug 5 03:41 src
drwxr-xr-x 2 avulasanjana avulasanjana 4096 Aug 5 03:41 tests
Child process completed.
```

Observation: I learned how Linux creates and manages processes using fork(). I understood that exec() can replace the child process with another program, while wait() allows the parent process to wait until the child process finishes. let's command.

SESSION-2

Task1: To Create Main Loop, Display Prompt, Read User Input, Handle Exit

Conditions, Design Control Flow Diagram, Test Interactive Loop

Source Code:



```
GNU nano 7.2 session2_task1.c
#include <stdio.h>
#include <string.h>

int main() {
    char command[100];

    printf("OSSP Interactive Shell\n");

    while (1) {
        printf("myshell> ");

        if (fgets(command, sizeof(command), stdin) == NULL) {
            break;
        }

        command[strcspn(command, "\n")] = '\0';

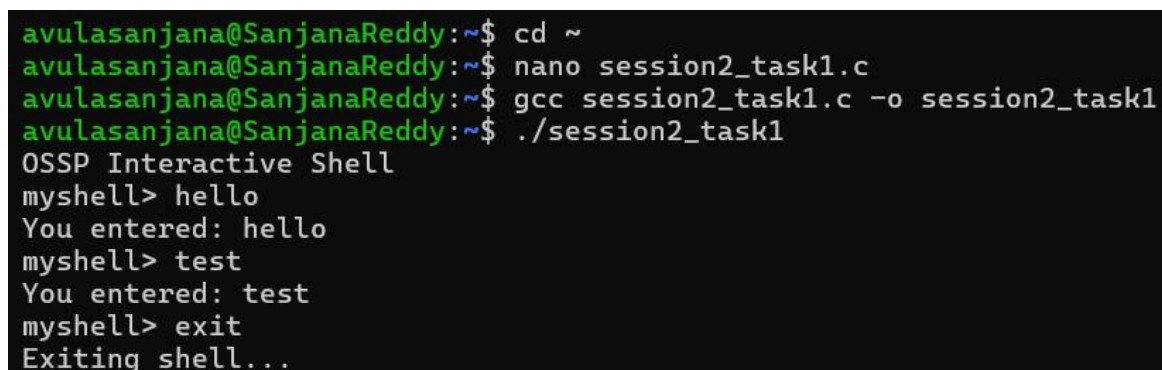
        if (strcmp(command, "exit") == 0) {
            printf("Exiting shell...\n");
            break;
        }

        if (strlen(command) == 0) {
            continue;
        }

        printf("You entered: %s\n", command);
    }

    return 0;
}
```

Output:



```
avulasanjana@SanjanaReddy:~$ cd ~
avulasanjana@SanjanaReddy:~$ nano session2_task1.c
avulasanjana@SanjanaReddy:~$ gcc session2_task1.c -o session2_task1
avulasanjana@SanjanaReddy:~$ ./session2_task1
OSSP Interactive Shell
myshell> hello
You entered: hello
myshell> test
You entered: test
myshell> exit
Exiting shell...
```

Observation: I learned how to create an interactive main loop in C that repeatedly displays a prompt and reads user input. I also learned how to handle exit

conditions, manage input strings, and control program flow using loops and conditional statements.

Task2: To Capture Keyboard Input, Handle Backspace, Process Enter Key, Manage Input Buffer, Support Multi-Character Commands, Test User Interaction.

Source code:

```
GNU nano 7.2
#include <stdio.h>

int main() {
    char buffer[100];
    int ch;
    int index = 0;

    printf("Type a command and press Enter:\n");
    printf("input> ");

    while ((ch = getchar()) != '\n' && ch != EOF) {
        if (ch == 127 || ch == 8) {
            if (index > 0) {
                index--;
            }
        } else if (index < 99) {
            buffer[index++] = ch;
        }
    }

    buffer[index] = '\0';

    printf("\nCommand entered: %s\n", buffer);
    printf("Characters stored: %d\n", index);

    return 0;
}
```

Output:

```
avulasanjana@SanjanaReddy:~$ nano session2_task2.c
avulasanjana@SanjanaReddy:~$ gcc session2_task2.c -o session2_task2
avulasanjana@SanjanaReddy:~$ ./session2_task2
Type a command and press Enter:
input> hello world

Command entered: hello world
Characters stored: 11
```

Observation: I learned how to capture keyboard input in C using `getchar()`. I understood how to manage an input buffer to store multiple characters, process the Enter key to finish input, and handle the Backspace key by removing the last character from the buffer. I also learned how user input can be processed and displayed as a complete command.